

Strategic Architecture Blueprint

Autonomous Agentic Control Plane

Status: Architecture & Planning Phase

1 Executive Summary (CEO-Ready Briefing)

We are building a centralized “digital office” to manage an autonomous AI workforce. This system connects our four primary technology assets—the yacht management software, the data collection backend, the automated insurance blog, and the internal human-relationship mapping tool—under one unified dashboard.

In business terms, this control plane allows specialized Artificial Intelligence (AI) “workers” to independently fix software errors, scrape data, and write code without constant human supervision. By automating these processes, we drastically reduce manual labor, accelerate our product development timelines, and create immense operational leverage.

To avoid the exorbitant costs usually associated with cloud-based AI computation (API token burn and server rental), we are executing a highly efficient **Hybrid Mesh Architecture**. We will run the heavy lifting (the AI reasoning and code execution) securely on our local 96GB Mac Studio, utilizing the cloud only for lightweight state management and the user interface.

Design Note on Visuals: For the initial builds, we are using a “tokenized” design system. This means we are focusing entirely on functionality first—ensuring data flows and agents communicate correctly. The styling will appear generic initially, but because we are using design tokens, our established brand sheet can be applied globally with a single update once the underlying mechanics are proven.

2 The Project Portfolio (The Scope of Work)

This automated workforce will be responsible for overseeing, integrating, and maintaining the following existing company assets:

- **CelesteOS:** Our primary yacht Property Management System (PMS). Tech stack: Supabase, Vercel, Cloudflare, Render, Sentry, Linear.
- **Celeste Data HQ:** Our backend data engine, hosted on Hetzner, responsible for scraping and collecting critical information (e.g., AIS tracking).
- **MYI (My Yachts Insurance):** An automated blogging and marketing platform hosted on Vercel with Google Tracking, designed to increase our digital footprint.
- **Influence:** Our internal relationship-mapping ontology tool used for human asset valuation, transaction metrics, and strategic targeting. Currently utilizing Vercel and

an Excel MVP database.

3 The Infrastructure Topology (The Hybrid Mesh)

This setup separates the "brain" (execution) from the "face" (UI) and the "memory" (database). By keeping the execution environment entirely on our Mac Studio, we avoid the immense friction of containerizing coding environments in the cloud, completely eliminate latency for local file operations, and maximize the ROI of our 96GB hardware investment.

3.1 1. The Brain: The Mac Studio (Local Engine Room)

Runs a custom Python or TypeScript daemon. It handles the actual multi-agent execution, local file system manipulation, and local LLM hosting. It securely coordinates all AI sub-workers.

3.2 2. The Memory: Hetzner Cloud (Persistent State & Ingress)

Hosts a PostgreSQL database (like a self-hosted Supabase instance) to store the agent ontology and a Redis queue to catch external webhooks and data scraping events.

3.3 3. The Face: Vercel (Control Plane UI)

A Next.js frontend rendering the graph-based ontology and a "Slack-equivalent" interface, connected directly to the database and the Mac via WebSockets.

3.4 4. The Secure Tunnel (The Bridge)

A Cloudflare Tunnel (cloudflared) or Tailscale daemon running on the Mac. This creates a secure outbound connection from the Mac Studio directly to the Next.js app or Hetzner server, allowing two-way WebSocket communication without opening any vulnerable router ports on our local network.

4 The Mac Studio Daemon (Execution & Automation)

With 96GB of unified memory, the Mac is uniquely positioned to handle both expensive API calls and "free" local meta-orchestration.

- **Headless Claude Code Multiplexing:** We will utilize Claude Code's non-interactive CLI mode. The daemon spawns isolated subprocesses for each agent role using the flags: `claude -p --output-format stream-json --dangerously-skip-permissions`. This streams real-time, structured JSON events back to our daemon instead of a human terminal, allowing us to parse exactly what the agent is doing at every step.
- **The Zero-Cost "CARL" Supervisor:** Instead of burning API credits on monitoring "context rot" (when an AI forgets its initial instructions over a long task), we load a quantized 70B parameter model (e.g., Llama 3 or Qwen) locally via Ollama or Apple's MLX framework. This local model constantly ingests the Claude Code JSON streams. When it detects cyclical reasoning or context bloat, it acts as the "CARL" orchestrator: halting the Claude CLI, summarizing the state into a compressed artifact, and injecting it into a freshly spawned agent.

- **Cron & Batch Jobs:** The local daemon runs a standard scheduler to trigger daily health checks, repository pulls, and system cleanups directly on the bare metal.

5 The Sentry-to-Agent Fault Pipeline

To handle external system faults asynchronously without crashing the local daemon, we are implementing a strict dispatch router.

1. **Ingress:** Sentry detects an error in CelesteOS and fires a webhook to an endpoint hosted on Hetzner.
2. **Queueing:** Hetzner strips the noise from the payload and pushes the raw stack trace into a Redis queue.
3. **Local Polling:** The Mac Studio daemon securely polls this queue over the tunnel.
4. **Dispatch:** When an error is pulled, the daemon queries the local database to find which agent node "owns" that specific code path. It spawns a new headless Claude Code instance inside that specific directory and injects the stack trace as the opening prompt.

6 The Vercel UI & "Agent Slack"

The frontend acts as the read/write window into the Mac Studio's terminal processes.

- **The Ontology Visualizer:** Using libraries like React Flow (potentially alongside GSAP or Three.js), we map the database schema into flattened nodes representing departments and agents, adapting the logic previously built for the Influence project.
- **Live WebSockets:** As the Mac Studio processes `--output-format stream-json data`, it pushes those events through the secure tunnel up to the Vercel app in real-time.
- **The Slack Interface:** Clicking on a node in the UI opens its specific communication channel. Human operators can view live terminal outputs, read lines of communication between Agent A and Agent B, or type a message. When a message is sent, it travels down the tunnel, and the local daemon injects it into the input stream of the running Claude Code subprocess.

7 Open-Source Forges & Integrations (Build vs. Buy)

To accelerate development, we will avoid reinventing the wheel by forking established enterprise protocols.

Developer Warning: Spot-check cloned repos for `stream-json` parsing capabilities and Model Context Protocol (MCP) adherence. Reject any open-source control planes that do not support MCP "Code Mode," which is required to prevent intermediate tool bloat.

- **Execution Layer:** Use Claude Code's headless CLI as detailed above.
- **Integration & Governance:** Implement open-source frameworks like the Agent Governance Toolkit (AGT) or Agent Control to evaluate tool calls, enforce role-based access policies, and prevent unauthorized prompt injection.

- **Graph Visualizer:** Fork React Flow to handle the drag-and-drop primitives of the departmental UI ontology.

8 The Custom Build Matrix

These components must be developed from scratch to glue the ecosystem together:

- **The CARL State Checkpointer:** The local MLX/Ollama logic monitoring JSON streams to detect context decay and inject handoff artifacts.
- **Sentry Dispatch Router:** The queuing mechanism mapping Hetzner traces to specific CelesteOS nodes.
- **Inter-Agent WebSocket Broker:** The deterministic JSON schema permitting worker agents across MYI and CelesteOS to pass variables securely to each other without data leakage or hallucinations.

9 Legacy System Audit (Jarvis / Kenki)

When auditing the legacy Jarvis/Kenki repository, autonomous agents and developers must scan for first-principles utility rather than forcing obsolete integrations.

- **Retain:** The Supabase schemas, ElevenLabs API hooks (for future voice integration), and established hierarchical topology definitions that map our company structure.
- **Refactor:** Strip out legacy context-rot logic that relies on full-history dumps. We are migrating exclusively to precise state extraction.

10 Component Status & File Ledger

- `mac_execution_daemon` (Status: To Be Created): The background program running on the Mac Studio. It is the engine of the entire workforce.
- `dashboard_ontology_ui` (Status: To Be Created): The visual frontend map translating AI operations into the chat interface.
- `carl_state_supervisor` (Status: To Be Created): The local oversight protocol (using the 70B model) that manages context rot.
- `legacy_jarvis_config` (Status: Reviewed / Needs Refactoring): The old AI setup containing valuable schemas but outdated logic.

11 Functionality, Risks & Immediate Next Steps

11.1 Identified Risks

Legacy code from the old AI setup may conflict with our new communication standards if not carefully filtered during the audit phase.

11.2 Action Plan

1. **Establish the Secure Tunnel:** Install and configure Cloudflare Tunnels (or Tailscale) on the Mac Studio to create the two-way connection to the Hetzner/Vercel cloud.
2. **Spin Up the UI Skeleton:** Deploy the Next.js Vercel dashboard with tokenized styling to serve as the visual testbed.
3. **Conduct a Data Flow Test:** Manually trigger an event on the Mac daemon and verify it accurately reflects in the cloud UI.